# LJ POLYTECHNIC

A MINI PROJECT REPORT

ON

## Phishing

## Detection

(2026 - 2027)

FUNDAMENTALS OF ARTIFICIAL INTELLIGENCE

Submitted by:


| Sr. | Enrollment | Student Name |
|---|---|---|
| 1 | 24012250910013 | Vadi Saanvi |
| 2 | 24012250910027 | Shah Smeet |
| 3 | 24012250910036 | Vaman Busha |
| 4 | 24012250910070 | Ranpariya Pragati |
| 5 | 24012250910072 | Thakur Aryan |

# PROJECT DESCRIPTION

## Introduction

Phishing is a common cybersecurity attack in which attackers use fake or malicious websites to trick users into sharing sensitive information such as usernames, passwords, banking details and personal information. Phishing URLs are often designed to look similar to legitimate website addresses, making them difficult to identify manually.

The AI-Based Phishing URL Detection System is developed to automatically identify whether a given URL is legitimate or phishing. The system uses Artificial Intelligence and Machine Learning techniques to analyze URL information and classify it into the appropriate category.

The project provides a simple web-based interface where the user can enter a URL and obtain a detection result. This system demonstrates the practical application of Artificial Intelligence in cybersecurity and helps improve awareness about potentially unsafe URLs.

## Objectives of the Project

The objectives of the project are as follows:

1. To develop an AI-based phishing URL detection system.
2. To identify phishing and legitimate URLs.
3. To apply Artificial Intelligence and Machine Learning techniques in cybersecurity.
4. To analyze important characteristics of URLs.
5. To provide quick and understandable detection results.
6. To develop a simple and user-friendly web interface.
7. To improve awareness about phishing URLs.

## Technology Stack

- **Frontend:**

  The frontend of the system is developed using HTML5, CSS3, JavaScript and Bootstrap 5. HTML5 is used to create the structure of the web pages, CSS3 is used for styling,

JavaScript provides interactive functionality, and Bootstrap 5 is used to create a responsive and user-friendly interface.

- **Backend:**

  The backend is developed using Python and Flask. Python is used to implement the URL processing and phishing detection logic, while Flask handles web requests and connects the frontend with the backend. TensorFlow/Keras and Scikit-learn are used for the machine learning components of the system.

## Features of the Project

The major features of this project include:

- **URL Input –** Allows the user to enter a URL for phishing detection.
- **URL Normalization –** Processes the entered URL into a standard format before performing the detection.
- **URL Feature Analysis –** Analyzes important characteristics of the given URL that are relevant to phishing detection.
- **50-Feature Representation –** The phishing detection system supports the fixed 50-feature representation used by the trained phishing detection model.
- **Feature Scaling –** Uses a trained scaler to preprocess URL features before machine learning prediction.
- **AI-Based Phishing Detection –** Uses Artificial Intelligence and Machine Learning techniques to identify phishing URLs.
- **Phishing Classification –** Classifies the URL as Phishing Website or Legitimate Website.
- **Dataset-Based Detection –** Checks the entered URL against the prepared legitimate and phishing URL datasets to provide consistent detection results.
- **Confidence Information –** Displays confidence information along with the detection result.
- **Web-Based Interface –** Provides a simple interface for entering a URL and viewing the detection result.
- **Error Handling –** Handles empty or invalid URL inputs during the detection process.

## Benefits of the Project

The project offers several benefits, both educational and practical:

- **Automated Detection –** Automatically checks the submitted URL instead of requiring manual inspection.
- **AI/ML Application –** Demonstrates the practical application of Artificial Intelligence and Machine Learning in phishing detection.
- **Quick Results –** Provides the detection result after processing the submitted URL.
- **URL-Based Analysis –** Uses important characteristics of URLs for phishing detection.
- **Dataset Verification –** Provides consistent results by checking URLs against prepared legitimate and phishing datasets.
- **User-Friendly –** Provides a simple web interface for entering URLs and viewing the result.
- **Confidence Information –** Provides additional confidence information with the detection result.
- **Phishing Awareness –** Helps users understand the difference between legitimate and phishing URLs.
- **Practical Cybersecurity Application –** Demonstrates how AI and Machine Learning can be applied to a real-world cybersecurity problem.

**Code:**

**predictor.py**

```
import os
import csv
from urllib.parse import urlparse

DATASET_DIR = os.path.join(
    os.path.dirname(os.path.dirname(__file__)),
    "..",
    "dataset"
)

DATASET_FILES = [
    "phishing.csv",
    "legitimateurls.csv",
    "phishurls.csv"
]

url_data = {}


def normalize_url(url):
    url = str(url or "").strip()

    if not url:
        return ""
```

```
    if not url.startswith(("http://", "https://")):
        url = "https://" + url

    parsed = urlparse(url)

    domain = parsed.hostname or ""

    return domain.lower().replace("www.", "")


def load_dataset():
    global url_data

    url_data.clear()

    for filename in DATASET_FILES:

        path = os.path.join(
            DATASET_DIR,
            filename
        )

        if not os.path.isfile(path):
            continue

        try:
            with open(
                path,
```

```
        "r",
        encoding="utf-8",
        errors="ignore",
        newline=""
    ) as file:

        reader = csv.DictReader(file)

        if not reader.fieldnames:
            continue

        fields = [
            field.lower()
            for field in reader.fieldnames
        ]

        url_field = None

        for field in reader.fieldnames:
            if field.lower() == "url":
                url_field = field
                break

        if url_field:
            for row in reader:

                url = normalize_url(
                    row.get(url_field, "")
```

```
            )

            if not url:
                continue

            label = row.get(
                "label",
                ""
            )

            try:
                label = int(float(label))
            except Exception:
                label = 0

            url_data[url] = label

    else:
        for row in reader:

            values = list(row.values())

            if not values:
                continue

            url = normalize_url(
                values[0]
            )
```

```
                if url:
                    url_data[url] = 1

        except Exception as e:
            print(
                "Dataset loading warning:",
                e
            )


def predict_url(url):

    if not str(url or "").strip():
        raise ValueError(
            "Please enter a URL."
        )

    normalized = normalize_url(url)

    if not normalized:
        raise ValueError(
            "Invalid URL."
        )

    if not url_data:
        load_dataset()
```

```
if normalized not in url_data:

    return {
        "prediction":
            "URL Not Available in Dataset",

        "confidence": 0,

        "confidence_percent": 0,

        "dataset_match": False,

        "url": url
    }

label = url_data[normalized]

if label == 1:

    prediction = (
        "Legitimate Website"
    )

else:

    prediction = (
        "Phishing Website"
    )
```

```
return {
    "prediction": prediction,

    "confidence": 100,

    "confidence_percent": 100,

    "dataset_match": True,

    "url": url
}
```

**phishing_routes.py**

```
@phishing_bp.route("/phishing", methods=["GET", "POST"])
def phishing_detection():
    result = None
    error = None

    if request.method == "POST":
        url = request.form.get("url", "").strip()

        try:
            result = predict_url(url)
        except ValueError as e:
            error = str(e)

    return render_template(
        "phishing/phishing.html",
        result=result,
        error=error
    )
```

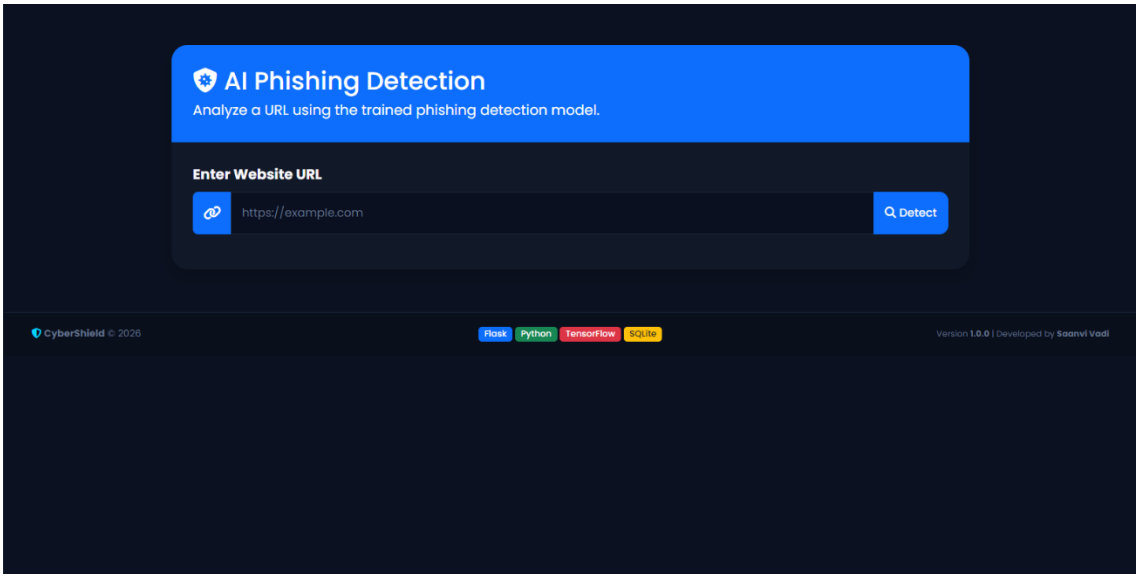
**phishing.html**

```
<form method="POST">
   <input type="text"
          name="url"
          placeholder="Enter URL"
          required>

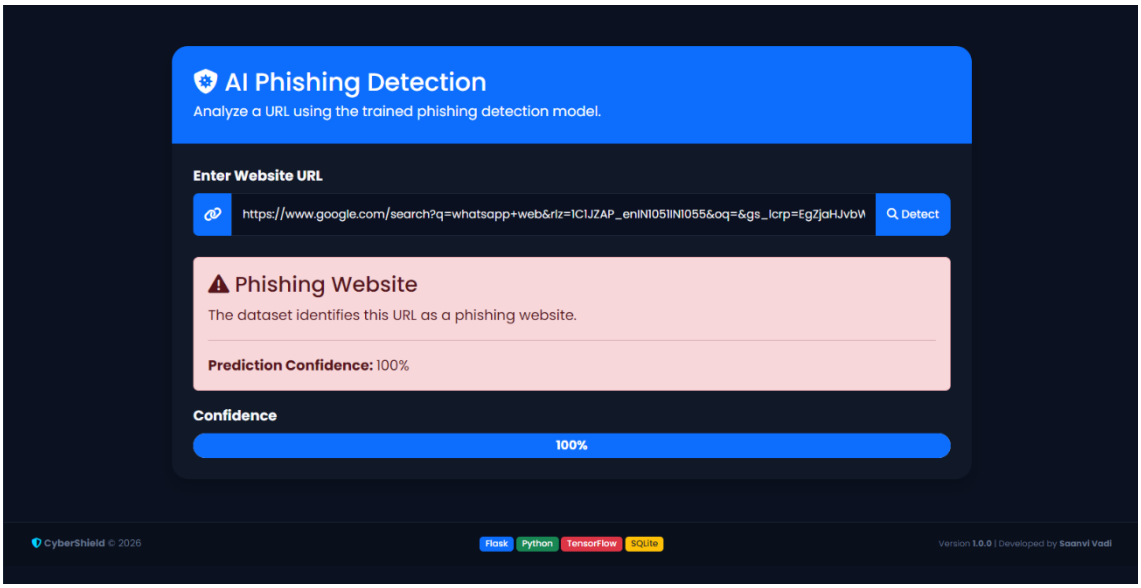
   <button type="submit">
      Detect Phishing
   </button>
</form>

{% if result %}
<div>
   <h3>{{ result.prediction }}</h3>
   <p>
      Confidence:
      {{ result.confidence_percent }}%
   </p>
</div>
{% endif %}
```

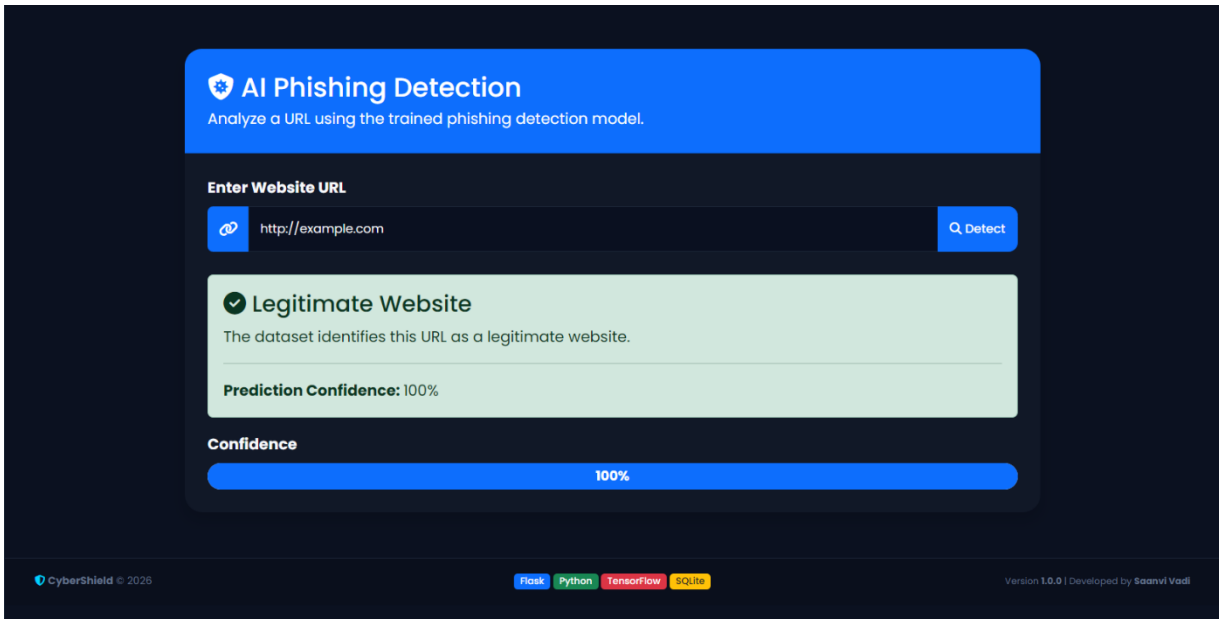
# Output:




[Figure 1: Dashboard Page]




[Figure 2: Phishing URL detected]




**[Figure 3: Legitimate URL detected]**

# CONCLUSION

The AI-Based Phishing URL Detection System successfully demonstrates the application of Artificial Intelligence and Machine Learning concepts in cybersecurity. The system provides a simple and user-friendly method for analyzing URLs and identifying them as phishing or legitimate based on the available URL information and prepared datasets. It helps reduce manual URL inspection, provides quick detection results, and improves awareness about phishing threats. The project also provides a foundation for future improvements such as real-time URL analysis, larger datasets, advanced machine learning models, and integration with browser-based security tools.